GPReg Documentation

Complete reference for the GPReg package.
For a quick overview, see `README.md`. For runnable examples, see `examples/demo.ipynb`.

Table of Contents

1 Installation **3**

2 Conceptual overview **3**

3 Models **3**
- 3.1 GaussianProcessRegressor . . . 3
- 3.2 SparseGPRegressor . . . 4
- 3.3 MultiOutputGP . . . 5

4 Kernels **6**
- 4.1 RBF . . . 6
- 4.2 Matern . . . 6
- 4.3 Linear . . . 7
- 4.4 White . . . 7
- 4.5 Composing kernels . . . 7

5 Preprocessing and pipelines **7**
- 5.1 StandardScaler . . . 7
- 5.2 PCA . . . 8
- 5.3 Pipeline . . . 8
- 5.4 make_gp_pipeline . . . 8

6 Diagnostics **8**
- 6.1 rmse(y_true, y_pred) . . . 9
- 6.2 nlpd(y_true, y_mean, y_std) . . . 9
- 6.3 loo_cv(gp_model) . . . 9

7 Plotting **9**

8 Persistence **10**

9 Exceptions **10**

10 Common workflows **10**
- 10.1 Workflow 1: 1D regression on small clean data . . . 10
- 10.2 Workflow 2: Numeric DataFrame with several features . . . 11
- 10.3 Workflow 3: Multivariate outputs . . . 11
- 10.4 Workflow 4: Large dataset (n in the thousands or more) . . . 11
- 10.5 Workflow 5: Custom kernel with PyTorch optimizer . . . 11
- 10.6 Workflow 6: Full diagnostic workflow . . . 12

11 Mathematical background **12**
11.1 Marginal likelihood . . . 12
11.2 Predictive distribution . . . 12
11.3 FITC sparse approximation . . . 13
11.4 Closed-form LOO-CV . . . 13

12 Performance and scaling **13**

13 Limitations **13**

14 References **14**

1 Installation

```
pip install -r requirements.txt
```

Or for a development install:

```
pip install -e .
```

The core dependencies are NumPy, SciPy, pandas, matplotlib, and seaborn. Three features are gated behind optional dependencies:

Feature	Dependency	Install with
K-means inducing point initialization	scikit-learn	`pip install -e .[sklearn]`
PyTorch autograd optimizer	torch	`pip install -e .[torch]`
Streamlit app	streamlit	`pip install -e .[app]`

To install everything: `pip install -e .[all]`.

2 Conceptual overview

A Gaussian Process (GP) is a distribution over functions. Instead of fitting a single curve to our data, a GP gives us a probability distribution over all possible curves consistent with your data. The mean function is then selected as our fitted curve and the kernel function furnishes us with uncertainty estimates.

A GP is fully specified by two things:

- A **mean function** (we use zero by default)
- A **kernel** (covariance function) that encodes assumptions about how outputs at nearby inputs should be correlated

The "training" step in this package is actually hyperparameter optimization: finding kernel parameters (length-scales, signal variance, noise variance) that maximize the marginal likelihood of the observed data.

Inputs and outputs must be continuous (numeric). GPReg does not support categorical variables; if your data has categorical features, you'll need to transform them to numeric (e.g., target encoding, learned embeddings) before passing them to GPReg.

For a deeper introduction to Gaussian Processes, the canonical reference is Rasmussen & Williams, *Gaussian Processes for Machine Learning* (2006), available free at gaussianprocess.org.

3 Models

3.1 GaussianProcessRegressor

Exact Gaussian Process Regression. Suitable for `n < ~5000` training points.

Constructor

```
GaussianProcessRegressor(
    kernel=None,          # Kernel; defaults to RBF() + White(0.1)
    normalize_y=True,     # Subtract mean of y before fitting
    n_restarts=5,         # Random restarts for hyperparameter optimization
```

```
    optimizer="L-BFGS-B", # "L-BFGS-B", any scipy method, or "pytorch"
    random_state=None,    # Seed for reproducibility
)
```

Methods

- `fit(X, y)`: Optimize kernel hyperparameters by maximizing marginal log-likelihood. Accepts numpy arrays or numeric pandas DataFrames; raises `ValueError` if non-numeric columns are present.
- `predict(X, return_std=False, return_cov=False)`: Predict means; optionally return standard deviations or full covariance.
- `sample_y(X, n_samples=1, random_state=None)`: Draw function samples from the posterior at points X.
- `score(X, y)`: Return R^2 on test data.

Attributes (after fitting)

- `X_train_`, `y_train_`: Stored training data
- `L_`: Cholesky factor of the kernel matrix
- `alpha_`: Solution to $K\alpha = y$, used for prediction
- `log_marginal_likelihood_`: Final log marginal likelihood
- `kernel`: Kernel with optimized hyperparameters

Example

```python
from gpreg import GaussianProcessRegressor, RBF, White
import numpy as np

X = np.linspace(-3, 3, 30).reshape(-1, 1)
y = np.sin(X).ravel() + 0.1 * np.random.randn(30)

gp = GaussianProcessRegressor(kernel=RBF() + White(0.1), random_state=0)
gp.fit(X, y)

X_test = np.linspace(-4, 4, 100).reshape(-1, 1)
y_mean, y_std = gp.predict(X_test, return_std=True)
samples = gp.sample_y(X_test, n_samples=5) # shape (100, 5)
```

3.2 SparseGPRegressor

Sparse GP using the FITC (Fully Independent Training Conditional) approximation. Reduces complexity from $O(n^3)$ to $O(nm^2)$, where m is the number of inducing points. Suitable for n in the tens of thousands since GP becomes undesirable when sample size becomes large.

Constructor

```
SparseGPRegressor(
    kernel=None,
    n_inducing=100,             # Number of inducing points (m)
    inducing_strategy="kmeans", # "kmeans", "random", or "subset"
```

```
    optimize_inducing=False,    # (not yet implemented; might do this over the summer)
    normalize_y=True,
    n_restarts=3,
    random_state=None,
)
```

Methods

Same `fit`, `predict`, `score` API as `GaussianProcessRegressor`. Note: `sample_y` and full predictive covariance are not implemented for sparse GPs.

Attributes

- `Z_`: Inducing point locations after fitting (shape `(n_inducing, d)`)
- All other attributes match the exact GP

When to use it

- $n >\sim 5000$ training points
- The function is reasonably smooth (FITC tends to over-smooth pathologically wiggly functions)
- You can tolerate a small accuracy hit in exchange for a large speedup

Example

```
from gpreg import SparseGPRegressor, RBF, White

sparse_gp = SparseGPRegressor(
    kernel=RBF() + White(0.1),
    n_inducing=100,
    inducing_strategy="kmeans",
)
sparse_gp.fit(X_large, y_large)
y_mean, y_std = sparse_gp.predict(X_test, return_std=True)
```

3.3 MultiOutputGP

Wraps any single-output GP (exact or sparse) to handle multivariate targets by training one independent GP per output dimension.

Important caveat: outputs are assumed conditionally independent given the inputs. For correlated outputs where information should be shared across dimensions, more advanced methods (Linear Model of Coregionalization, multi-task kernels) would be appropriate but are not implemented.

Constructor

```
MultiOutputGP(base_gp) # base_gp is deep-copied per output
```

Methods

- `fit(X, Y)`: Y must be numeric: a 2D array `(n, p)`, a numeric DataFrame, or a Series.
- `predict(X, return_std=False)`: Returns `(m, p)` shaped arrays.
- `score(X, Y)`: Mean R^2 across outputs.

- `per_output_scores(X, Y)`: Array of R^2 per output.

Attributes

- `estimators_`: List of one fitted GP per output
- `n_outputs_`: Number of output dimensions
- `output_names_`: Column names if Y was a DataFrame, else None
- `log_marginal_likelihood_`: Sum across outputs

Example

```
from gpreg import GaussianProcessRegressor, MultiOutputGP, RBF, White
import pandas as pd

Y = pd.DataFrame({
    "temperature": temp_values,
    "humidity":   humid_values,
})

base = GaussianProcessRegressor(kernel=RBF() + White(0.1))
mgp = MultiOutputGP(base)
mgp.fit(X, Y)

Y_mean, Y_std = mgp.predict(X_new, return_std=True)
print(mgp.per_output_scores(X, Y))
```

4 Kernels

Kernels live in `gpreg.kernels`. All kernels inherit from `Kernel` and support composition via `+` (sum) and `*` (product).

4.1 RBF

Radial Basis Function (Squared Exponential) kernel. Produces very smooth (infinitely differentiable) functions.

$$k(x, x') = \sigma^2 \exp\left(-\frac{\|x - x'\|^2}{2\ell^2}\right)$$

```
RBF(length_scale=1.0, signal_var=1.0)
```

The default kernel choice when you don't have a strong reason to pick something else.

4.2 Matern

Matérn kernel with smoothness parameter ν. Less smooth than RBF, often a better fit for real-world data.

```
Matern(length_scale=1.0, signal_var=1.0, nu=2.5)
```

Supported ν values: `0.5` (exponential, very rough), `1.5` (once-differentiable), `2.5` (twice-differentiable, most popular). ν is treated as fixed and is not optimized.

4.3 Linear

Linear kernel for modeling linear trends. Equivalent to Bayesian linear regression on the inputs.

$$k(x, x') = \sigma^2(x \cdot x')$$

```
Linear(signal_var=1.0)
```

Most useful as an additive component combined with a smooth kernel, e.g., `Linear() + RBF()`.

4.4 White

White noise kernel. Adds independent observation noise.

```
White(noise_var=1.0)
```

Almost always used as the additive noise component in a composite kernel: `RBF() + White(0.1)`.

4.5 Composing kernels

```
from gpreg import RBF, Matern, Linear, White

# Trend + smooth structure + noise
k1 = Linear() + RBF() + White(0.1)

# Product of kernels
k2 = RBF(length_scale=5.0) * Matern(length_scale=0.5, nu=1.5)

# Multi-component kernel
k3 = (RBF() + Matern(nu=2.5)) * Linear() + White(0.05)
```

The hyperparameters of all sub-kernels are optimized jointly during `fit()`.

5 Preprocessing and pipelines

Live in `gpreg.preprocessing`.

5.1 StandardScaler

Standardize features to zero mean and unit variance. **Always scale your inputs before fitting a GP.** Kernel length-scales become meaningless when features have wildly different ranges.

```
from gpreg import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_test_scaled = scaler.transform(X_test)
X_back = scaler.inverse_transform(X_scaled) # round-trip
```

Constant columns (zero variance) are detected and left alone instead of producing NaN.

5.2 PCA

Principal Component Analysis via SVD for dimension reduction.

```
from gpreg import PCA

pca = PCA(n_components=0.95) # keep 95% of variance
# pca = PCA(n_components=5) # or keep exactly 5 components
X_proj = pca.fit_transform(X)

print(pca.explained_variance_ratio_)
print(pca.n_components_)
```

Useful for high-dimensional inputs where the curse of dimensionality hurts the GP. Computed via SVD rather than covariance eigendecomposition for numerical stability when features are nearly collinear.

5.3 Pipeline

Chain transformers and a GP estimator together.

```
from gpreg import Pipeline, StandardScaler, PCA, GaussianProcessRegressor, RBF, White

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("pca",  PCA(n_components=0.95)),
    ("gp",   GaussianProcessRegressor(kernel=RBF() + White(0.1))),
])
pipe.fit(X_train, y_train)
y_mean, y_std = pipe.predict(X_test, return_std=True)
```

Access individual steps via `pipe.named_steps["scale"]` or `pipe.estimator` for the final GP.

5.4 make_gp_pipeline

Convenience factory for the standard preprocessing pipeline:

```
from gpreg import make_gp_pipeline, GaussianProcessRegressor, RBF, White

pipe = make_gp_pipeline(
    GaussianProcessRegressor(kernel=RBF() + White(0.1)),
    scale=True,             # add StandardScaler
    pca_components=None,    # add PCA if not None
)
```

This is the simplest way to get a complete preprocessing-plus-GP pipeline.

6 Diagnostics

Live in `gpreg.diagnostics`. Three core scalar metrics for assessing model quality.

6.1 rmse(y_true, y_pred)

Standard root mean squared error. Lower is better!

6.2 nlpd(y_true, y_mean, y_std)

Negative log predictive density. It evaluates the full predictive distribution against the truth. Penalizes both inaccurate means *and* poorly calibrated uncertainty: a confidently-wrong prediction is punished more than an uncertain wrong one. Lower is better. Reported per data point (averaged), so values are comparable across datasets.

6.3 loo_cv(gp_model)

Leave-one-out cross-validation using the closed-form formula (Rasmussen & Williams, Eq. 5.12). Avoids the $O(n^4)$ cost of naive LOO by reusing the already-computed Cholesky factor. Returns a dict with:

- `loo_means`: predictive mean at each training point with that point left out
- `loo_vars`: predictive variance at each training point with that point left out
- `loo_rmse`: overall LOO RMSE
- `loo_nlpd`: overall LOO NLPD

Only available for the exact GP, not the sparse GP.

```
from gpreg import loo_cv

results = loo_cv(gp)
print(f"LOO RMSE: {results['loo_rmse']:.4f}")
print(f"LOO NLPD: {results['loo_nlpd']:.4f}")
```

7 Plotting

Live in `gpreg.diagnostics`. All functions return `(fig, ax)` so you can customize, save, or compose them with other plots.

Function	Purpose
`plot_predictions_1d(gp, X_test, ...)`	Mean + uncertainty band + training points + optional posterior samples
`plot_predictions_2d(gp, x1_range, x2_range, plot_type="mean")`	Heatmap of predictive mean or std over a 2D grid
`plot_residuals(y_true, y_mean, y_std=None)`	Standardized residuals vs predicted (or raw if no std given)
`plot_kernel_heatmap(kernel, X)`	Visualize the kernel matrix as a seaborn heatmap
`plot_pair(gp, X_test, feature_names=None)`	Predicted-y vs each input feature with others held at median

Example:

```
from gpreg.diagnostics import plot_predictions_1d, plot_residuals
import matplotlib.pyplot as plt
```

```
fig, ax = plot_predictions_1d(gp, X_test, n_samples=3, true_fn=lambda x: np.sin(x))
ax.set_title("My GP fit")
plt.show()
```

8 Persistence

```
from gpreg import save_model, load_model

save_model(gp, "my_model.pkl")       # pickle (default)
save_model(gp, "my_model.json", format="json") # JSON for inspection only

loaded = load_model("my_model.pkl")
```

Pickle is recommended for round-trip save/load. JSON saves only the configuration (kernel hyperparameters, training data, settings) and is useful for inspecting model parameters in a human-readable format, but JSON loading currently raises `NotImplementedError` because reconstructing arbitrary composite kernels from text would require a full parser.

9 Exceptions

All in `gpreg.utils`. All inherit from `GPRegError`.

Exception	When raised
`KernelDimensionError`	Input dimensions don't match between training and test data, or between the two arguments to a kernel call
`NonPSDMatrixError`	Kernel matrix is not positive definite even after adding maximum jitter (often caused by duplicate input points)
`ConvergenceError`	All hyperparameter optimization restarts failed
`NotFittedError`	`predict()` or related method called on an unfitted model
`InvalidHyperparameterError`	A hyperparameter is outside its valid range (e.g., negative length-scale)

In addition, GP models raise `ValueError` when given DataFrames with non-numeric columns.

```
from gpreg.utils import NonPSDMatrixError

try:
    gp.fit(X, y)
except NonPSDMatrixError:
    print("Try adding jitter or removing duplicate points.")
```

10 Common workflows

10.1 Workflow 1: 1D regression on small clean data

```
from gpreg import GaussianProcessRegressor, RBF, White
from gpreg.diagnostics import plot_predictions_1d
```

```
gp = GaussianProcessRegressor(kernel=RBF() + White(0.1), random_state=0)
gp.fit(X, y)
fig, ax = plot_predictions_1d(gp, X_test, n_samples=3)
```

10.2 Workflow 2: Numeric DataFrame with several features

```
from gpreg import make_gp_pipeline, GaussianProcessRegressor, RBF, White

pipe = make_gp_pipeline(
    GaussianProcessRegressor(kernel=RBF() + White(0.1), n_restarts=5),
    scale=True,
)
pipe.fit(df_train, y_train)
y_mean, y_std = pipe.predict(df_test, return_std=True)
```

10.3 Workflow 3: Multivariate outputs

```
from gpreg import GaussianProcessRegressor, MultiOutputGP, RBF, White

base = GaussianProcessRegressor(kernel=RBF() + White(0.1))
mgp = MultiOutputGP(base)
mgp.fit(X_train, Y_train)      # Y_train is numeric DataFrame or 2D array
Y_mean, Y_std = mgp.predict(X_test, return_std=True)
```

10.4 Workflow 4: Large dataset (n in the thousands or more)

```
from gpreg import SparseGPRegressor, RBF, White

sparse_gp = SparseGPRegressor(
    kernel=RBF() + White(0.1),
    n_inducing=200,
    inducing_strategy="kmeans",
)
sparse_gp.fit(X_large, y_large)
```

10.5 Workflow 5: Custom kernel with PyTorch optimizer

```
gp = GaussianProcessRegressor(
    kernel=RBF() + Matern(nu=1.5) + White(0.1),
    optimizer="pytorch",
    n_restarts=3,
)
gp.fit(X, y)
```

10.6 Workflow 6: Full diagnostic workflow

```python
from gpreg import rmse, nlpd, loo_cv
from gpreg.diagnostics import plot_residuals
import matplotlib.pyplot as plt

# Fit on train, predict on held-out test
gp.fit(X_train, y_train)
y_pred, y_std = gp.predict(X_test, return_std=True)

# Scalar metrics
print(f"RMSE: {rmse(y_test, y_pred):.4f}")
print(f"NLPD: {nlpd(y_test, y_pred, y_std):.4f}")

# Closed-form leave-one-out on training set
loo = loo_cv(gp)
print(f"LOO RMSE: {loo['loo_rmse']:.4f}")

# Residual diagnostic
fig, ax = plot_residuals(y_test, y_pred, y_std)
plt.show()
```

11 Mathematical background

11.1 Marginal likelihood

The objective optimized during `fit()` is the log marginal likelihood (Rasmussen & Williams Eq. 2.30):

$$\log p(y \mid X, \theta) = -12y^\top K^{-1}y - 12\log|K| - n2\log(2\pi)$$

where K is the kernel matrix at the training inputs (with hyperparameters θ) plus noise. We minimize the negative of this with respect to θ.

In practice the inverse and determinant are never computed directly. We instead use a Cholesky decomposition $K = LL^\top$, then:

- Solve $LL^\top\alpha = y$ via two triangular solves (gives $K^{-1}y$)
- $\log|K| = 2\sum_i \log(L_{ii})$ (avoids overflow)

This is in `gpreg.utils.linalg` and is the numerical backbone of the package.

11.2 Predictive distribution

Given a fitted GP with Cholesky factor L and $\alpha = K^{-1}y$, the predictive distribution at test points X^* is Gaussian with:

$$mean = K(X^*, X)\alpha$$

$$covariance = K(X^*, X^*) - K(X^*, X)K^{-1}K(X, X^*)$$

Both are computed without forming K^{-1} explicitly, using the same Cholesky machinery.

11.3 FITC sparse approximation

The FITC method (Snelson & Ghahramani 2006) approximates the full kernel matrix K with a low-rank-plus-diagonal structure built from $m \ll n$ inducing points Z:

$$K \approx Q_{nn} + \Lambda$$

$$Q_{nn} = K_{nm} K_{mm}^{-1} K_{mn} \quad (rankm)$$

$$\Lambda = diag(K_{nn} - Q_{nn}) + \sigma^2 I \quad (preservesmarginalvariancesexactly)$$

The Woodbury matrix identity then lets us evaluate the marginal likelihood and predictive distribution in $O(nm^2)$ instead of $O(n^3)$.

11.4 Closed-form LOO-CV

Naive leave-one-out for a GP would require refitting the model n times ($O(n^4)$ total). Rasmussen & Williams (Eq. 5.12) gives an exact closed form using only the already-computed K^{-1}:

$$\mu_i^{loo} = y_i - \frac{\alpha_i}{[K^{-1}]_{ii}}$$

$$\sigma_i^{2,loo} = \frac{1}{[K^{-1}]_{ii}}$$

This drops the cost to $O(n^2)$ per evaluation given the existing Cholesky factor.

12 Performance and scaling

Approximate fit times on a modern laptop (single CPU thread):

Method	**n = 100**	**n = 1000**	**n = 5000**	**n = 10000**
Exact GP	<1s	∼10s	∼5min	impractical
Sparse GP (m=100)	<1s	∼2s	∼10s	∼30s
Sparse GP (m=200)	<1s	∼5s	∼30s	∼2min

These are with `n_restarts=3` and the default optimizer. The exact GP scales as $O(n^3)$ per likelihood evaluation; sparse GP scales as $O(nm^2)$.

For the SciPy vs PyTorch optimizers: SciPy's L-BFGS-B with analytical computation is typically faster on small-to-medium problems for our standard kernels. The PyTorch backend wins when (a) you've defined a custom kernel that's hard to differentiate analytically, or (b) you can run on a GPU (not currently configured but trivial to enable).

13 Limitations

Things this package does *not* do, that you should know about if your application needs them:

- **Categorical inputs or outputs.** GPReg accepts continuous (numeric) data only. If your problem has categorical features, transform them to numeric before passing them in (target encoding, learned embeddings, or a separate categorical model that you combine with the GP for residuals).
- **Genuinely correlated multi-output GPs.** `MultiOutputGP` fits independent GPs per output. For sharing information across outputs (e.g., when one output is much more densely observed than another), you would want a Linear Model of Coregionalization or a multi-task kernel.

- **Non-Gaussian likelihoods.** This is a regression-only package. Classification, count data (Poisson), and other non-Gaussian likelihoods would require approximate inference (Laplace, EP, variational), which isn't implemented.
- **Scalable inducing-point optimization.** `SparseGPRegressor` fixes inducing point locations after initialization rather than optimizing them jointly with hyperparameters.
- **GPU support.** The PyTorch backend is set up to make this easy but doesn't currently move tensors to GPU.
- **Streaming / online learning.** All training data must fit in memory.

If you hit any of these limits, GPyTorch (https://gpytorch.ai/) is a great mature library that handles them all.

14 References

- Rasmussen, C. E. & Williams, C. K. I. (2006). *Gaussian Processes for Machine Learning.* MIT Press. Free at gaussianprocess.org.
- Snelson, E. & Ghahramani, Z. (2006). Sparse Gaussian Processes using Pseudo-inputs. *NIPS 2006.*
- Bauer, M., van der Wilk, M., & Rasmussen, C. E. (2016). Understanding Probabilistic Sparse Gaussian Process Approximations. *NIPS 2016.* Discusses tradeoffs between FITC, DTC, and VFE.